Verify Element Chat Encryption Using Wireshark

1. OBJECTIVE:

To Determine whether Element's network traffic is encrypted or transmitted in plaintext by capturing and analyzing packets with Wireshark.

2. Environment:

The testing environment contains the following components:

- Wireshark (to display the network traffic)
- Npcap (Network Packet Capture – for capturing the packets over the network)
- Element Web (two existing accounts are used here in this task).

3. Methodology:

Following steps are taken to perform the task:

- Two existing accounts of Element Web is use in this task. Messages are send from one account to another in an end-to-end encrypted chatbox.
- To setup the environment first Wireshark is installed in the system to display the findings of this task (i-e network traffic to be captured).
- And to capture the traffic of network (messages that are being sent via Element chat box),Npcap is installed in system.
- After that the settings are made for displaying the Wi-Fi traffic only, at Wireshark.
- Messages are sent in the chatbox and simultaneously the capture option is selected to capture the encrypted traffic.
- After sending the all text messages capturing is stopped to analyze the results.
- Four display filters were applied over the captured traffic that are: tls, http, tcp.port == 443, and frame contains "msgtype", to analyze and prove whether the traffic was encrypted or not

3.1 Findings:

The stated filters that were applied are discussed here one by one here to show the findings and understanding of results of these filters:

3.1.1 TLS Filter:

The major purpose of applying TLS filter is to show only the traffic that is encrypted, i-e the packets that are using TLS protocol and following a secure connection.

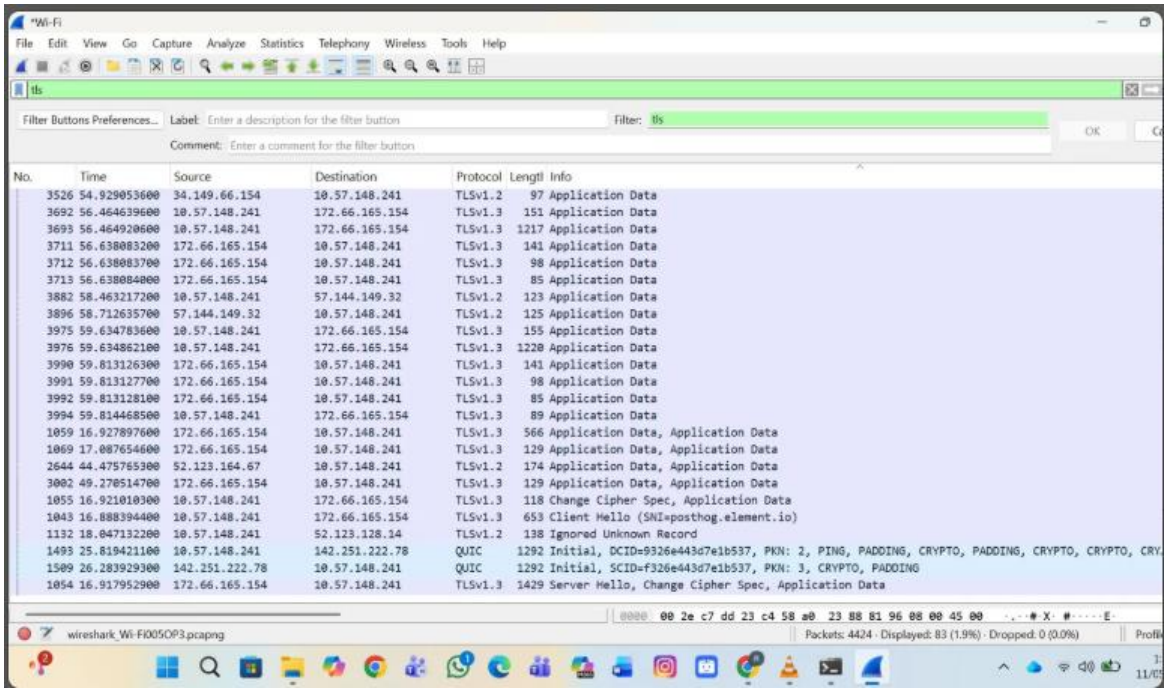

Figure 1: TLS filter results screenshot

- The above results shows **TLSv1.2** and **TLSv1.3** everywhere which shows that data packets are encrypted.
- The **"Application data"** written in **"info"** column shows that message contents are not visible and encryption is working.
- **"Client Hello (SNI=posthog.element.io)"** and **"Server Hello, Change Cipher Spec, Application Data"** shows the establishment of TLS handshake between client and server.

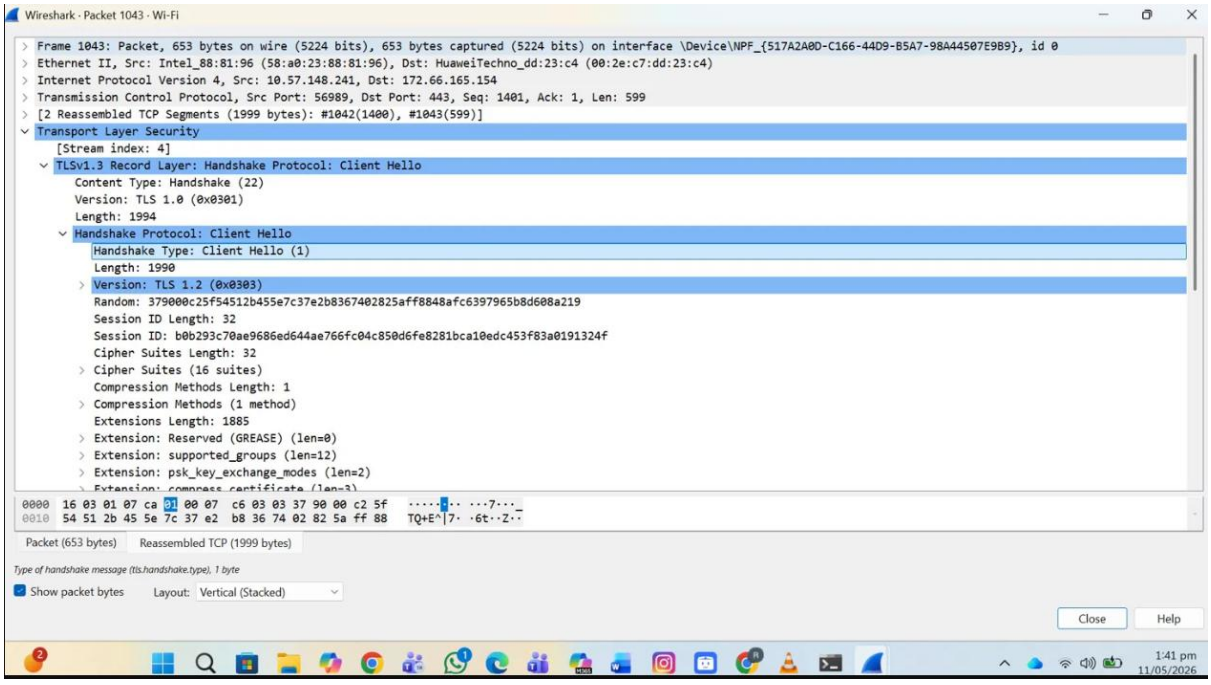

*Figure 2: Detailed screenshot of **"Client Hello"***

- The presence of **hex data** at the bottom shows that messages are encrypted and are in **unreadable format** and here is the prove.

- The **cipher suits details** shows the encryption algorithm list that are all **strong algorithms**. And the details in this screenshot is proving that it is the **traffic of "Element"**.

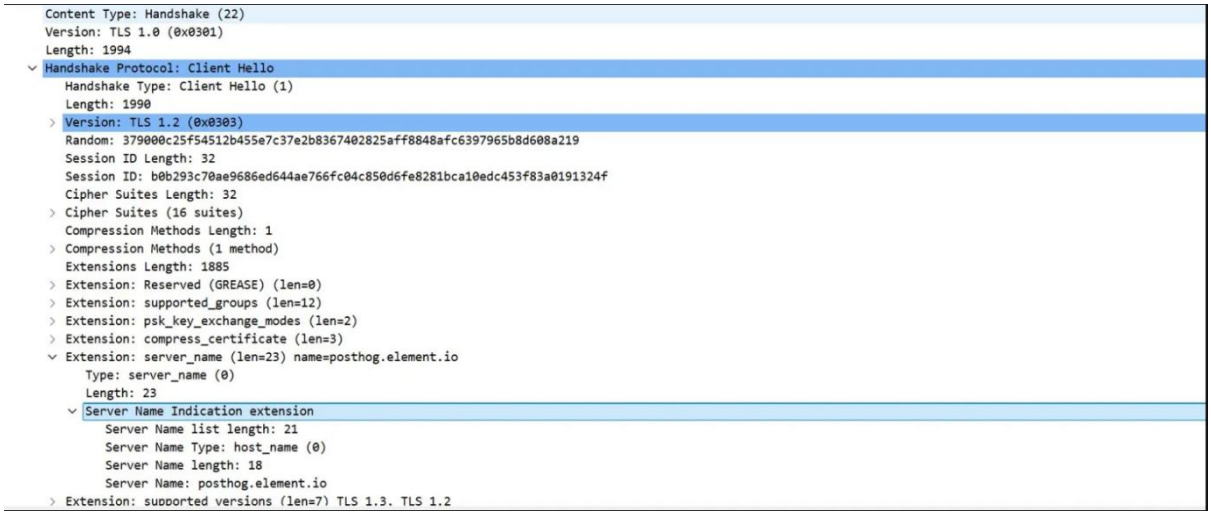

Figure 3: SNI Details

- **"Extension: server_name" "(len=23)" "name=posthog.element.io" "Server Name: posthog.element.io"** these details confirms the details of which packet is expended is Element's data packet as SNI is a TLS extension use in HTTPS connections.

3.1.2 HTTP Filter:

HTTP filter is set to capture the traffic that is transmitted over unsecured network at which the traffic is not encrypted.

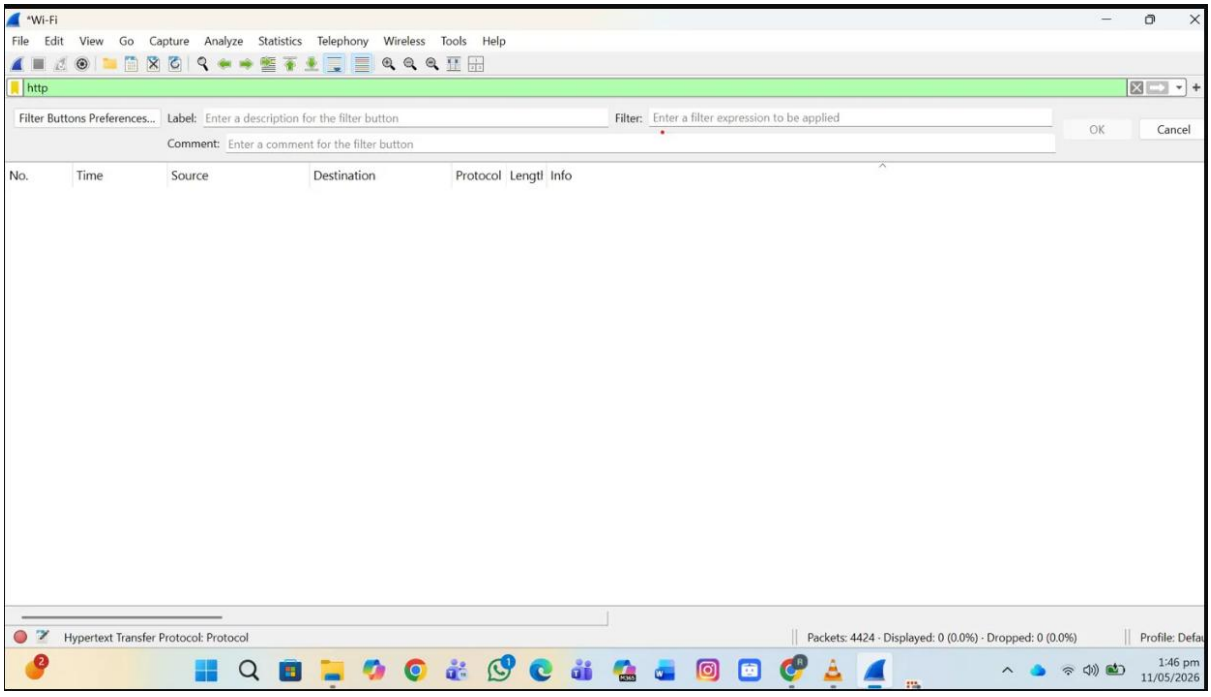

Figure 4: HTTP filter results screenshots

- The above screen is empty which shows that no unencrypted traffic is send over the network and the Element traffic is 100% encrypted and secured by HTTPS and TLS.

3.1.3 TCP Port filter:

TCP port filter shows only that specific port traffic which is sett for the system.

- TCP port == 443 shows the HTTPS port traffic that shows the encrypted traffic only.

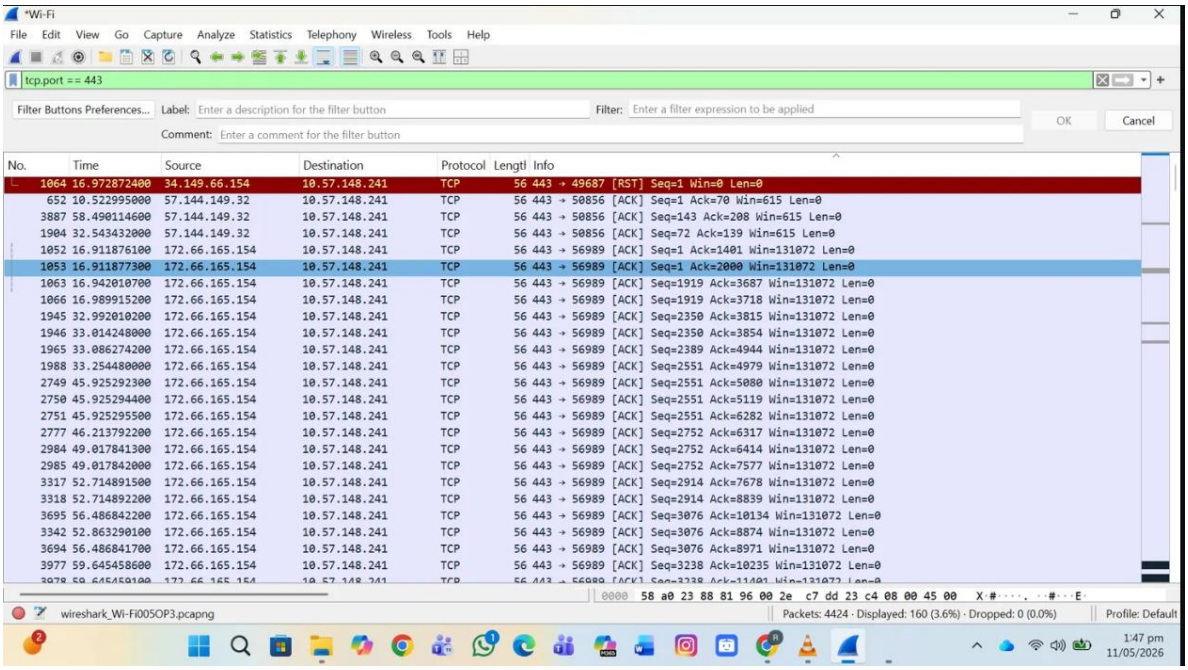

Figure 5: PORT=443 filtertraffic

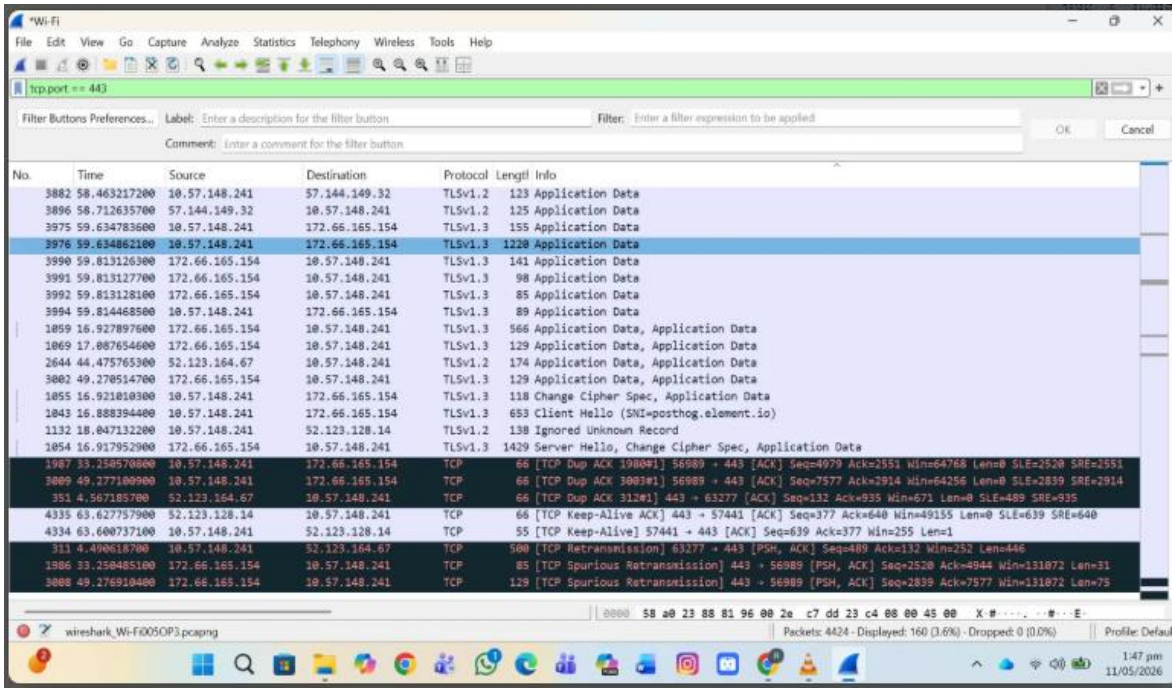

Figure 6: TLS port == 443 filter details.

3.1.4 Frame contains filter:

frame contains "xyz" filter is use to search the word written in inverted commas. If the packets are not encrypted this filter searches for the respective word in each data packet and then display that packet on Wireshark display section.

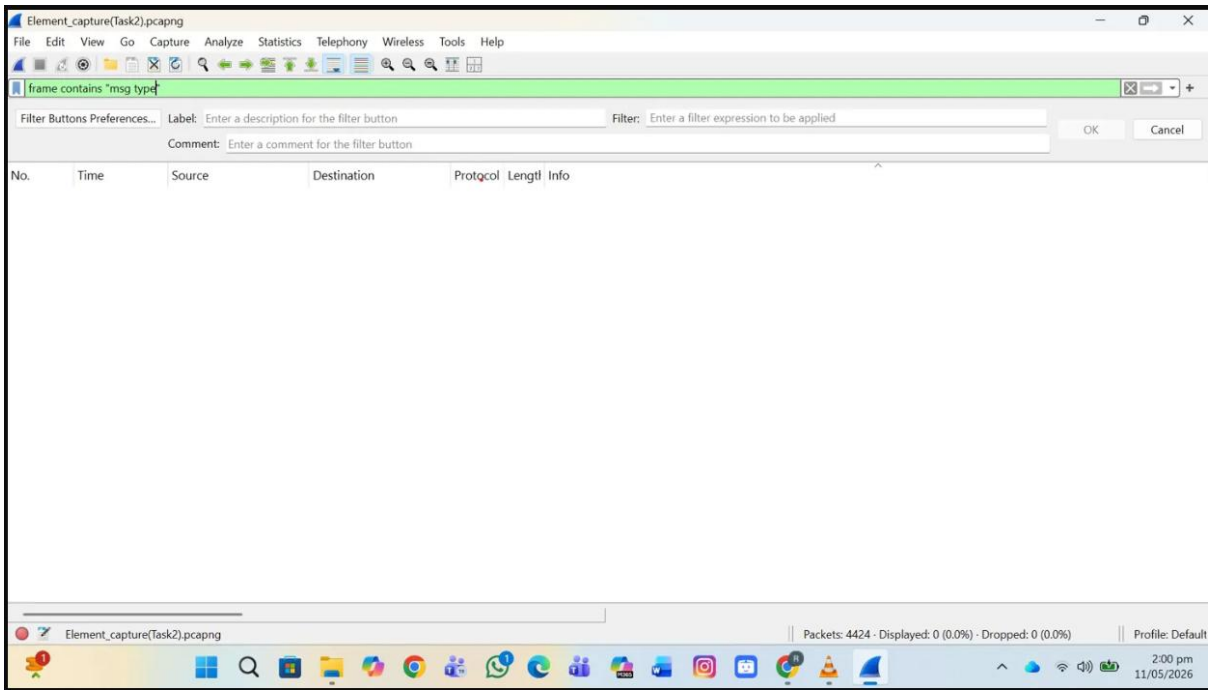

Figure 7: frame contains "msg type" filter

- This filter frame contain "message type" shows an empty screen is the proof that the searching for this word is impossible in the captured network traffic of Element – as all data packets were encrypted.
- No **"msgtype" is readable in data packets**
- And no Matrix message field is visible in **plaintext.**

4. Conclusion:

The analysis confirms that Element's network traffic is fully encrypted at the transport layer using TLS 1.2 and TLS 1.3 on port 443. No readable message content, HTTP traffic, or plaintext data was found in any captured packet. This proves that Element uses secure encrypted communication. Additionally, Element also supports End-to-End Encryption (E2EE) at the application layer, which means even the server cannot read the messages.

5. Review Questions:

1. **Was the traffic HTTP or HTTPS? How can you tell?**
   The TLS traffic was HTTPs as the captured traffic was found on port 443 and no HTTp packets were found when search by filter.
2. **What TLS version was negotiated?**
   TLS 1.3 was the primary version negotiated, with some connections using TLS 1.2. This was visible in the TLS handshake packets in Wireshark.

3. **Could you read any message content in the payload?**
   No message contents are not visible as all messages were encrypted. All packets showed only encrypted "Application Data" in hex/binary format and no plaintext was visible.
4. **What's the difference between TLS encryption and end-to-end encryption (E2EE)?**
   TLS encrypts traffic between the user and the server — so the server can still read the messages. E2EE encrypts messages from sender to receiver directly — even the server cannot read them
5. **What would be a red flag that encryption is broken or missing?**
   If HTTP traffic appeared on port 80, message content was readable in the payload, or no TLS layer was present in the captured packets — these would all indicate broken or missing encryption.